OS Lab

Skill-18

To Analyze Signal Delivery, Understand Signal Masks, Review Process Groups, Explore Terminal Signals, Test Signal Behavior.

To implement how to resume Stopped Jobs, Send Continue Signals, Update Job Status, Maintain Job Table, Verify Execution, Test Background Recovery.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

#define MAX_JOBS 20

typedef struct
{
   int job_id;
   pid_t pid;
   pid_t pgid;
   char command[100];
   char state[20];
} Job;

Job jobs[MAX_JOBS];
int job_count = 0;
int next_job_id = 1;

/* Find a job using its job ID */
int find_job(int job_id)
{
   for (int i = 0; i < job_count; i++)
   {
      if (jobs[i].job_id == job_id)
      {
         return i;
      }
   }
```

```
    return -1;
}

/* Add a job to the job table */
void add_job(pid_t pid, pid_t pgid, char *command)
{
    if (job_count >= MAX_JOBS)
    {
        printf("Job table is full\n");
        return;
    }

    jobs[job_count].job_id = next_job_id++;
    jobs[job_count].pid = pid;
    jobs[job_count].pgid = pgid;

    strcpy(jobs[job_count].command, command);
    strcpy(jobs[job_count].state, "Stopped");

    job_count++;
}

/* Remove a job */
void remove_job(int index)
{
    for (int i = index; i < job_count - 1; i++)
    {
        jobs[i] = jobs[i + 1];
    }

    job_count--;
}

/* Check current status of all jobs */
void update_job_status()
{
    int status;

    for (int i = 0; i < job_count; i++)
    {
        pid_t result = waitpid(
            jobs[i].pid,
            &status,
```

```
            WNOHANG | WUNTRACED
        );

        if (result == jobs[i].pid)
        {
            if (WIFEXITED(status))
            {
                printf(
                    "\n[%d] Completed: %s\n",
                    jobs[i].job_id,
                    jobs[i].command
                );

                remove_job(i);
                i--;
            }
            else if (WIFSIGNALED(status))
            {
                printf(
                    "\n[%d] Terminated: %s\n",
                    jobs[i].job_id,
                    jobs[i].command
                );

                remove_job(i);
                i--;
            }
            else if (WIFSTOPPED(status))
            {
                strcpy(
                    jobs[i].state,
                    "Stopped"
                );
            }
        }
    }
}

/* Display all jobs */
void list_jobs()
{
    update_job_status();

    if (job_count == 0)
```

```
    {
        printf("No jobs available\n");
        return;
    }

    printf("\n");
    printf("Job ID\tPID\tPGID\tState\tCommand\n");
    printf("-----------------------------------------------\n");

    for (int i = 0; i < job_count; i++)
    {
        printf(
            "[%d]\t%d\t%d\t%s\t%s\n",
            jobs[i].job_id,
            jobs[i].pid,
            jobs[i].pgid,
            jobs[i].state,
            jobs[i].command
        );
    }

    printf("\n");
}

/* Resume a stopped job */
void resume_job(int job_id)
{
    int index = find_job(job_id);

    if (index == -1)
    {
        printf("No such job: %d\n", job_id);
        return;
    }

    Job *job = &jobs[index];

    printf(
        "Resuming job [%d]: %s\n",
        job->job_id,
        job->command
    );

    /*
```

```
 * Send SIGCONT to the entire process group.
 * Negative PGID means the signal is sent
 * to the process group.
 */
if (kill(-job->pgid, SIGCONT) == -1)
{
   perror("kill(SIGCONT)");
   return;
}

strcpy(job->state, "Running");

printf(
   "[%d] Status changed to Running\n",
   job->job_id
);

/*
 * Wait for the process to finish or stop again.
 */
int status;

waitpid(
   job->pid,
   &status,
   WUNTRACED
);

if (WIFEXITED(status))
{
   printf(
      "[%d] Completed\n",
      job->job_id
   );

   remove_job(index);
}
else if (WIFSIGNALED(status))
{
   printf(
      "[%d] Terminated\n",
      job->job_id
   );
```

```
        remove_job(index);
    }
    else if (WIFSTOPPED(status))
    {
        strcpy(
            job->state,
            "Stopped"
        );

        printf(
            "[%d] Stopped again\n",
            job->job_id
        );
    }
}

/* Execute a command */
void execute_command(char *input)
{
    char command_copy[100];

    strcpy(command_copy, input);

    char *args[20];
    int argc = 0;

    char *token = strtok(
        input,
        " \t\n"
    );

    while (
        token != NULL &&
        argc < 19
    )
    {
        args[argc++] = token;

        token = strtok(
            NULL,
            " \t\n"
        );
    }
```

```
args[argc] = NULL;

if (argc == 0)
{
   return;
}

pid_t pid = fork();

if (pid < 0)
{
   perror("fork");
   return;
}

if (pid == 0)
{
   /*
    * Create a new process group.
    */
   setpgid(0, 0);

   /*
    * Restore default signal behavior.
    */
   signal(SIGINT, SIG_DFL);
   signal(SIGTSTP, SIG_DFL);
   signal(SIGCONT, SIG_DFL);

   execvp(
      args[0],
      args
   );

   perror("execvp");
   exit(EXIT_FAILURE);
}

/*
 * Parent creates the same process group.
 */
setpgid(pid, pid);

printf(
```

```
    "Started process PID=%d PGID=%d\n",
    pid,
    pid
);

/*
 * Wait until process finishes
 * or is stopped using Ctrl+Z.
 */
int status;

waitpid(
    pid,
    &status,
    WUNTRACED
);

if (WIFSTOPPED(status))
{
    printf(
        "\nProcess stopped by signal %d\n",
        WSTOPSIG(status)
    );

    add_job(
        pid,
        pid,
        command_copy
    );

    printf(
        "[%d] Stopped    %s\n",
        jobs[job_count - 1].job_id,
        jobs[job_count - 1].command
    );
}
else if (WIFEXITED(status))
{
    printf(
        "Process completed with status %d\n",
        WEXITSTATUS(status)
    );
}
else if (WIFSIGNALED(status))
```

```
    {
        printf(
            "Process terminated by signal %d\n",
            WTERMSIG(status)
        );
    }
}

int main()
{
    char input[100];

    printf("=== Job Resume and Signal Demo ===\n");
    printf("Commands:\n");
    printf("  jobs       - list stopped jobs\n");
    printf("  resume N   - resume job N\n");
    printf("  exit       - exit program\n");
    printf("\n");

    while (1)
    {
        update_job_status();

        printf("resume$ ");
        fflush(stdout);

        if (fgets(
                input,
                sizeof(input),
                stdin
            ) == NULL)
        {
            break;
        }

        input[
            strcspn(input, "\n")
        ] = '\0';

        if (strlen(input) == 0)
        {
            continue;
        }
```

```
        if (strcmp(input, "exit") == 0)
        {
            break;
        }

        if (strcmp(input, "jobs") == 0)
        {
            list_jobs();
            continue;
        }

        if (strncmp(input, "resume ", 7) == 0)
        {
            int job_id = atoi(input + 7);

            resume_job(job_id);

            continue;
        }

        execute_command(input);
    }

    return 0;
}
```

```
lahari@DESKTOP-AIC33F7:~$ cd /home/coder/project
nano resume_jobs.c
-bash: cd: /home/coder/project: No such file or directory
lahari@DESKTOP-AIC33F7:~$ gcc resume_jobs.c -o resume_jobs
lahari@DESKTOP-AIC33F7:~$ ./resume_jobs
=== Job Resume and Signal Demo ===
Commands:
  jobs       - list stopped jobs
  resume N   - resume job N
  exit       - exit program

resume$ sleep 30
Started process PID=429 PGID=429
^Z
[1]+  Stopped                 ./resume_jobs
lahari@DESKTOP-AIC33F7:~$ jobs
[1]+  Stopped                 ./resume_jobs
lahari@DESKTOP-AIC33F7:~$ resume 1
resume: command not found
lahari@DESKTOP-AIC33F7:~$ jobs
[1]+  Stopped                 ./resume_jobs
lahari@DESKTOP-AIC33F7:~$
```